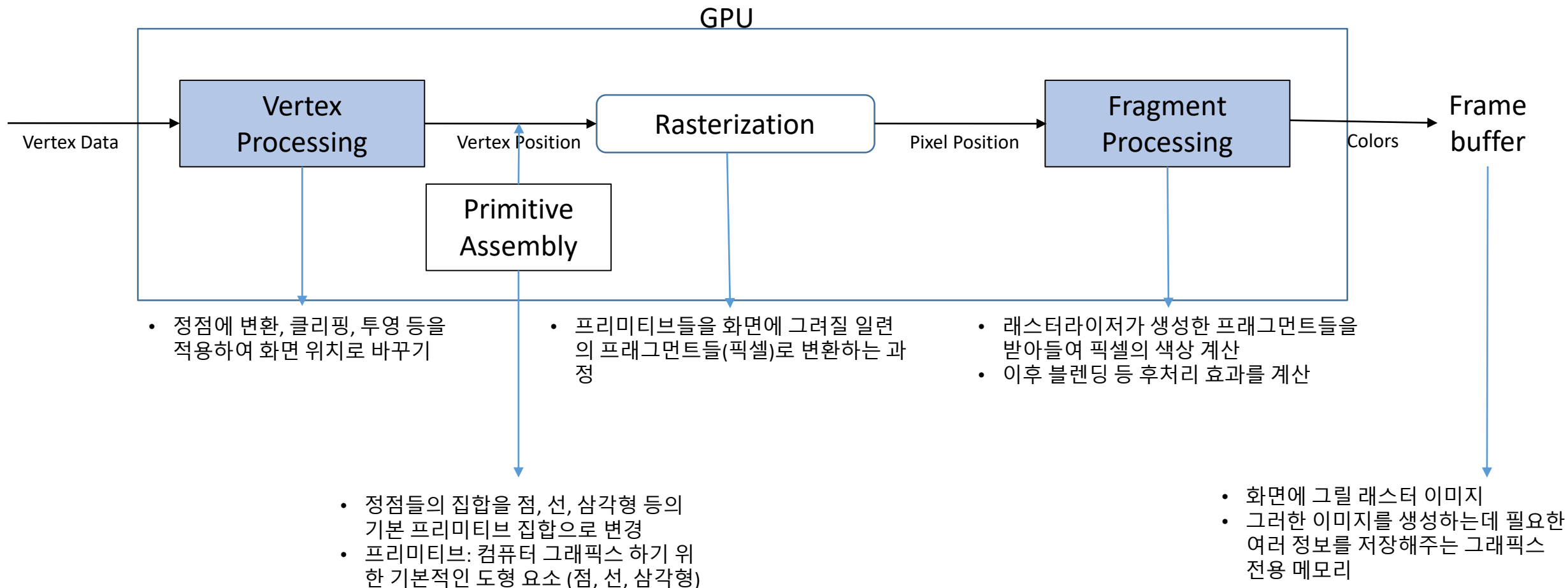


OpenGL 셰이더 사용하기

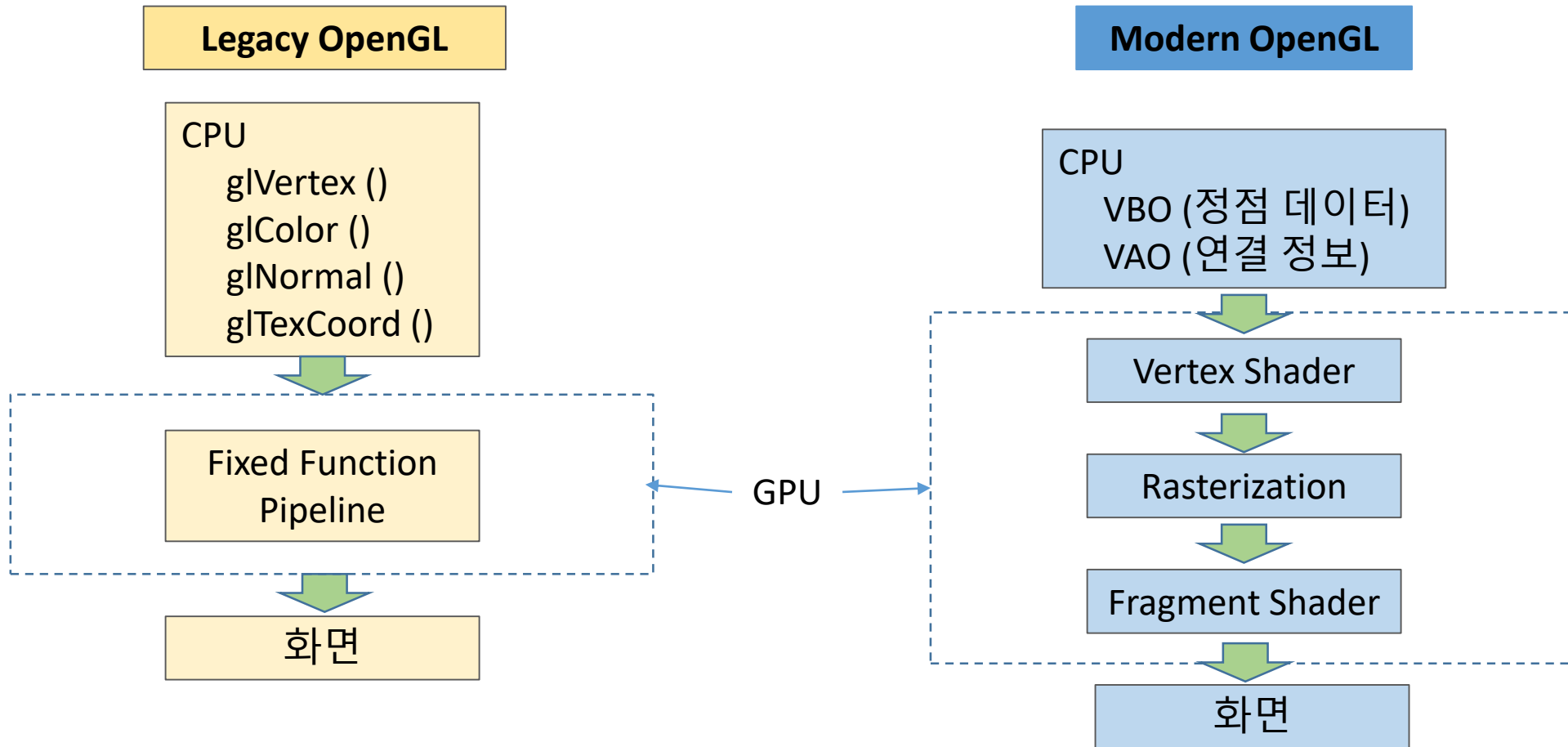
2026년 2학기

그래픽스 파이프라인

- 그래픽스 파이프라인
 - 그래픽스 파이프라인은 GPU가 그림을 그리는 과정
 - 즉 3차원 공간에 존재하는 물체의 데이터를 받아 화면에 보여주기 위한 이미지 데이터를 만들어가는 처리 과정



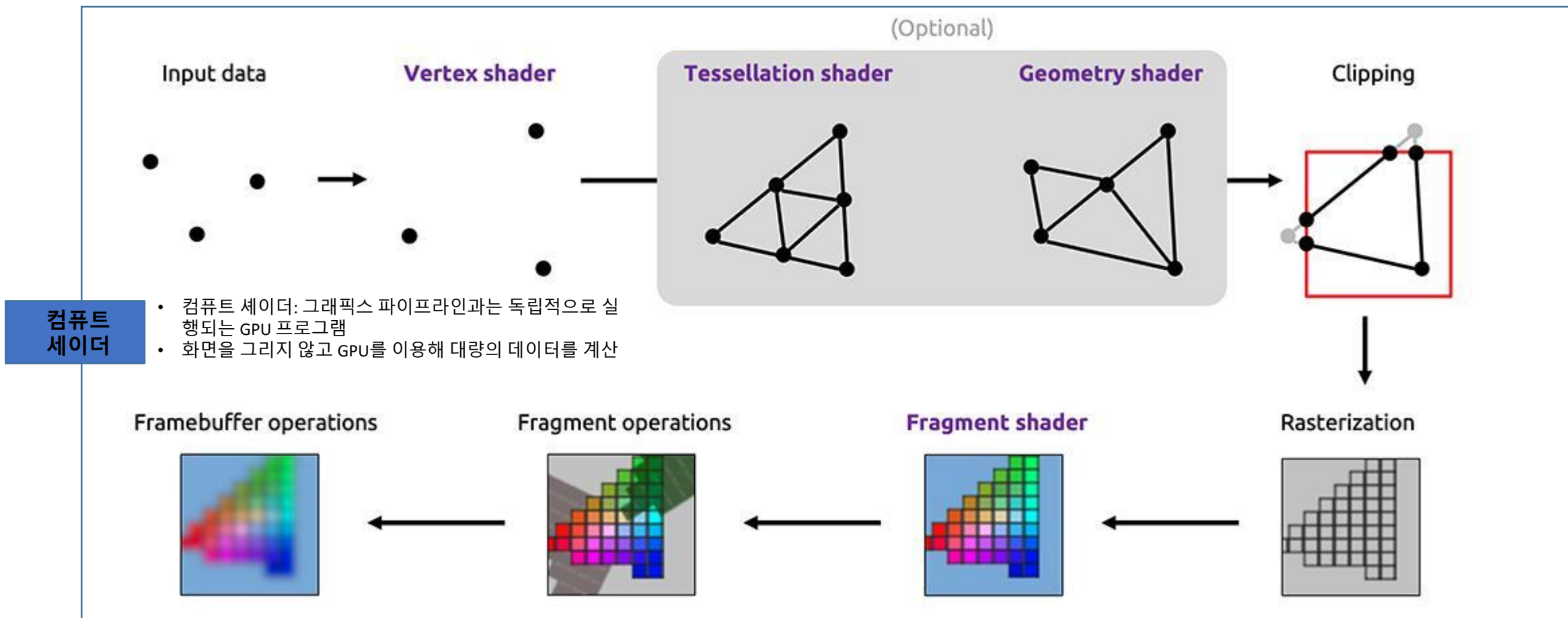
그래픽스 파이프라인



- Legacy OpenGL: OpenGL이 제공하는 고정된 그래픽스 처리 기능을 호출하여 사용했지만,
- Modern OpenGL: VBO/VAO에 데이터를 준비하고 GLSL 셰이더를 작성하여 GPU의 처리 과정을 프로그래머가 직접 정의한다.

그래픽스 파이프라인

- **버텍스 셰이더**: 첫 번째 프로그래밍 가능한 스테이지로 유일한 필수 스테이지
- 정점 처리: 버텍스 값, 노말 값, 텍스처 값 등
- **테셀레이션 셰이더**: 삼각형을 더 작은 삼각형으로 세분화 하는 셰이더
- 지형, 산, 곡면 등을 부드럽게 만들어준다.
- **지오메트리 셰이더**: 기본 도형을 이루는 버텍스들의 모음을 입력받아 다음 셰이더 단계에 보내기 전에 적절한 형태로 변환



- 프레임버퍼 동작: 화면에 출력
- 프레임버퍼: 화면에 보이는 영역, 색상 이외에도 픽셀당 값을 저장
- **프래그먼트 셰이더**: 프래그먼트의 색상을 결정, 프레임 버퍼로 전달
- 라이팅, 재질, 프래그먼트 깊이 결정하는 일을 수행
- 래스터라이제이션: 프래그먼트들이 어떤 선이나 삼각형에 의해 채워지는지 결정

OpenGL 그래픽스 파이프라인

- 모던 오픈지엘 주요 단계

셰이더	입력	출력	역할
Vertex Shader	정점(Vertex)	변환된 정점	위치 계산, 좌표 변환
Tessellation Shader	패치(Patch)	세분화된 정점	메시 세분화
Geometry Shader	점·선·삼각형	새로운 도형	도형 생성 및 확장
Fragment Shader	프래그먼트(Fragment)	픽셀 색상	색상, 텍스처, 조명 계산
Compute Shader	데이터	계산 결과	범용 GPU 계산(GPGPU) (General-Purpose GPU)

- 이번 학기는 오픈지엘 3.3 이용하여 Vertex shader와 Fragment shader 구현

- 두 shader 코드들은 메인 프로그램과는 분리해서 각각 따로 작성한다.
- 모든 정점 (vertex)와 속성 (attribute) 정보를 배열(array)로 지정한다.
 - Vertex buffer Object와 Vertex Array Object 사용
- 이 배열을 GPU에 보내어 저장하고 렌더링에 사용한다.

- 오픈지엘 버전을 3.3으로 지정

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);  
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);  
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

셰이더 작성하기

- openGL 셰이더: GLSL로 작성한다.
 - **GLSL (openGL Shading Language) :**
 - 오픈지엘 프로그램 안에서 GPU에서 실행되는 셰이더를 작성하기 위한 프로그래밍 언어
 - 많은 데이터를 동시에 처리하는 GPU의 병렬 처리 구조를 전제로 설계된 언어
 - 셰이더 종류에 따라 입력과 출력이 다르다.
 - C언어를 기초로 한 상위 레벨 셰이딩 언어
 - GLSL에는 포인터 개념이 없음: 동적 할당이 없음
 - 함수 오버로딩을 지원
 - 재귀 호출이 허용되지 않음
 - C언어의 구조체 사용 가능
 - 행렬(Matrices)과 벡터(Vectors) 타입을 기본 자료형으로 제공
 - 벡터 연산을 위한 다양한 함수 제공
 - GLSL의 컴파일러는 openGL에 내장되어 있음

오픈지엘 함수
그래픽스 파이프라인을
프로그래머가 제어하기
위한 API

GLSL 셰이더
그래픽스 파이프라인의
특정 단계를 프로그래머
가 작성하는 프로그램

GLSL 기본 문법

- 데이터 타입
 - 기본 C언어 스칼라 타입: **int, float, double, uint, bool**
 - 벡터 타입: **vec** → 위치, 색상, 방향, 법선 등에 사용
 - 지원하는 모든 스칼라 타입에 대한 벡터 타입 지원,
 - 접두어: 성분의 기본 자료형, 접미어: 인자 개수, ($2 \leq n \leq 4$)
 - **vecn / dvecn / bvecn / ivec n / uvec n**
 - 예) **vec4 : float 4개, ivec3: int 3개, uint2: uint 2개**
 - Vec2, vec3, vec4: 2D, 3D, 4D float-타입 벡터
 - 벡터의 구성요소 접근은 대응되는 구조체로 접근 가능
 - 좌표값: x, y, x, w 로 사용
 - 색상값: r, g, b, a로 사용
 - 텍스처 좌표: s, t, p, q로 사용
 - 벡터의 요소들은 배열처럼 접근 가능
 - 예) **vec4 foo; float x = foo[0]; float y = foo[1];...**
 - 스위izzling (swizzling): 여러 성분을 한번에 선택할 수 있다.

GLSL 기본 문법

- 데이터 타입
 - 행렬 타입: **mat**
 - float와 double 타입에 대한 행렬 지원
 - 접미어에서 앞의 숫자가 열, 뒤 숫자가 행 ($2 \leq n \leq 4, 2 \leq m \leq 4$)
 - **mat***n* / **mat***m**n* / **dmat***n* / **dmat***n**m*
 - mat2, mat3, mat4: 2x2, 3x3, 4x4 매트릭스(행렬)
 - 예) mat3x2: 3열 2행 행렬
 - 행렬은 배열처럼 다룰 수 있고, 열 우선으로 구성
 - 텍스처 샘플러: **sampler**
 - 텍스처 값 (texel)에 접근이 가능한 샘플러 타입
 - **sampler1D, sampler2D, sampler3D**

GLSL 기본 문법

- 생성자 (Constructor)
 - 변수의 초기화는 C++ 생성자 방식 이용
 - `vec3 aPos = vec3 (0.0f, 0.5f, 0.0f);`
 - 벡터 생성자는 저장할 값의 숫자를 지정한다.
 - `vec3 position (1.0);` `//--- vec3 position (1.0, 1.0, 1.0);`
 - 벡터에 하나의 스칼라값을 지정하면 벡터의 모든 요소에 할당
 - `vec4 whiteColor = vec4 (1.0f);` `//--- vec4 whiteColor (1.0, 1.0, 1.0, 1.0);`
 - 스칼라와 벡터, 행렬을 생성자 내에서 혼합해 사용할 수 있다.
 - `vec4 aColor = vec4 (r, vec2(g, b), a);` `//--- vec4 aColor (r, g, b, a)`
 - 배열 생성자는 다음의 문법에 따라 생성자를 호출한다.
 - `const float array[3] = float[3] (2.5, 7.0, 1.5);`
 - `const vec4 vertex[3] = vec4[3] (vec4(0.25, -0.25, 0.5, 1.0), vec4(-0.25, -0.25, 0.5, 1.0), vec4(0.25, 0.25, 0.5, 1.0));`
 - 행렬은 열 우선으로 구성되고, 단일 스칼라 값을 지정하는 경우 대각 행렬이 됨 (대각 요소 외에는 0으로 채워짐)
 - `mat3 m = mat3 (`
 `1.1, 2.1, 3.1,       //--- 첫번째 열`
 `1.2, 2.2, 3.2,       //--- 두번째 열`
 `1.3, 2.3, 3.3);       //--- 세번째 열`
 - `mat3 m = mat3 (1.0);` `//--- 대각선이 1.0이고 다른 요소들은 0.0인 3x3 행렬`
 - 변환은 $M \cdot v$ 로 표현 (행렬 * 벡터)

GLSL 기본 문법

- 연산자

- C 언어에서 사용하는 대부분의 연산자가 지원된다.

- 산술 연산자: +, -, *, /, %

- 비트 연산자: &, |, ^, ~, <<, >>

- 논리 연산자: !, &&, ||

- 비교 연산자: <, <=, >, >=, ==, !=

- 삼항 연산자: ?

- 대입 연산자: =, +=, -=

- 대부분의 연산자는 C언어의 우선순위 규칙과 동일하게 연산자 사용 가능

- 이항 연산일 경우, 벡터 연산은 요소별 연산으로 진행

- 예)

```
vec3 a = vec3(1.0, 2.0, 3.0);  
vec3 b = vec3(0.1, 0.2, 0.3);  
vec3 c = a + b;           // = vec3(1.1, 2.2, 3.3)  
vec3 d = a * b;           // = vec3(0.1, 0.4, 0.9)
```

- 예)

```
mat2 a = mat2(1., 2., 3., 4.);  
mat2 b = mat2(10., 20., 30., 40.);  
mat2 c = a * b;           // = mat2( 1. * 10. + 3. * 20., 2. * 10. + 4. * 20.,  
                           //      1. * 30. + 3. * 40., 2. * 30. + 4. * 40.)
```

- 예)

```
vec3 a = vec3(1.0, 2.0, 3.0);  
mat3 m = mat3(1.0);  
float s = 10.0;  
vec3 b = s * a;           // vec3(10.0, 20.0, 30.0)  
vec3 c = a * s;           // vec3(10.0, 20.0, 30.0)  
mat3 m2 = s * m;         // = mat3(10.0)  
mat3 m3 = m * s;         // = mat3(10.0)
```

GLSL 기본 문법

- 예) 행렬-벡터 곱셈 (벡터는 행렬의 오른쪽에서 곱셈 진행해야 함)

```
vec2 v = vec2(10., 20.);
```

```
mat2 M = mat2(1., 2., 3., 4.);
```

```
vec2 w = M * v;
```

```
// = vec2(1. * 10. + 3. * 20., 2. * 10. + 4. * 20.)
```

$$Mv = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} m_{11}v_1 + m_{12}v_2 \\ m_{21}v_1 + m_{22}v_2 \end{bmatrix}$$

GLSL 기본 문법

- 연산자

- 비트 연산은 허용되지 않음
- 스위즐(swizzling) 연산자 (C의 선택 연산자(.))
 - 행렬 및 벡터형으로부터 복수의 구성 요소들을 선택
 - [] 또는 . 을 이용하여 벡터 및 행렬 요소에 접근 가능

- vec4 타입의 변수는 각각

- x, y, z, w

→ 좌표값으로 사용할 때

- 또는 r, g, b, a

→ 색상으로 사용할 때

- 또는 s, t, p, q

→ 텍스처 좌표값으로 사용할 때

의 요소로 사용가능하다

- `vec4 a = vec3 (0.0f, 1.0f, 2.0f, 3.0f);`

`//--- a[2] == a.z == a.b == a.p` → 아무 요소로나 사용 가능

- 요소 선택자를 이용하여 재배치 및 복제, 일부 요소 수정 가능

- `vec4 a;`

- `vec4 b = a.xyxx;`

- `vec3 c = b.zwx;`

- `vec4 d = a.xxyy + b.yxzy;`

GLSL 기본 문법

- 한정자 (qualifier)
 - 한정자는 변수나 함수의 형식 인자의 앞에 붙여 해당 변수나 인자의 특성을 결정짓는다.
 - 한정자 + 자료형 + 변수명
 - 한정자 종류: const, in/out, uniform, layout...
 - 예) in vec3 position; //--- 한정자: in, 자료형: vec3, 변수명: position
- 한정자 (qualifier)
 - const
 - 상수
 - 셰이더에 의해 실행 중에 변수가 변경되는 것을 방지
 - 사용 예)
 - const float one = 1.0;
 - const vec3 origin = vec3 (1.0, 2.0, 3.0);

GLSL 기본 문법

- 한정자 (qualifier)

- in/out

- 입력/출력 변수로 CPU로부터 데이터를 전달받고 각 셰이더의 연산 결과를 전달할 때 사용
 - 입력 변수는 이전 셰이더 스테이지와 연결, 출력 변수는 다음 셰이더 스테이지로 연결
 - in: 입력 변수로 셰이더 안으로 들어오는 데이터. 함수 내부에서 읽기만 가능, 기본 한정자
 - out: 출력 변수로 셰이더 밖으로 전달하는 데이터. 함수 시작 시점에는 정의되지 않은 값을 가지며, 함수 종료 시점에 가진 값을 호출자에게 전달함 (함수 내부에서 호출자로의 쓰기만 가능)
 - 연결된 스테이지의 입력-출력 변수는 같은 이름으로 사용

- 사용 예)

- //--- vertex shader

```
in vec3 position;           //--- CPU로부터 받는 정점 속성
const vec4 red = vec4 (1.0, 0.0, 0.0, 0.0);
out vec3 color;             //--- 프래그먼트 셰이더로 보내기

void main ()
{
    gl_Position = vec4 (position, 1.0);
    color = red;
}
```

- //--- fragment shader

```
in vec3 color;              //--- 버텍스 셰이더에서 받기
out vec4 outColor;          //--- 프레임 버퍼로 보내기

void main ()
{
    outColor = vec4 (color, 1.0);
}
```

GLSL 기본 문법

- 한정자 (qualifier)

- layout

- 데이터의 위치나 배치 방법을 지정하는 한정자
 - 1개 이상의 속성을 가진 데이터들이 저장된 버퍼의 속성 순서를 설정
 - 형태: **layout (layout 지정) 변수 선언**
 - layout 지정 (layout specification):
 - location: 셰이더 입력/출력 변수의 번호 지정
 - 사용 예) layout (location = 0)
 - 필요에 따라 layout () 안에 여러 개의 레이아웃 지정을 사용할 수도 있다.
 - 사용 예) 버텍스 셰이더에서 layout을 이용하여 2개의 속성 변수 설정하기

```
#version 330 core
layout (location = 0) in vec3 vPos;           //--- 위치 변수: attribute position 0
layout (location = 1) in vec3 vColor;        //--- 컬러 변수: attribute position 1

out vec3 outColor;                            //--- 컬러를 fragment shader로 전달

void main()
{
    gl_Position = vec4(aPos, 1.0);
    outColor = vColor;                        //--- 컬러를 outColor에 설정
}
```

GLSL 기본 문법

- 한정자 (qualifier)
 - uniform: CPU의 응용프로그램에서 GPU의 셰이더로 데이터를 전달하는 한 방법
 - 모든 단계의 셰이더에서 공통적으로 사용하는 읽기 전용 데이터
 - 필요한 셰이더에서 전역 변수 형태로 선언한 후 사용
 - 셰이더가 아니라 응용 프로그램에서 값을 설정할 수 있고, 셰이더에서는 디폴트 값으로 초기화할 수 있다.
 - 리셋을 하거나 업데이트를 하기 전까지 그 값을 계속 유지
 - Uniform을 설정할 때는 어떤 셰이더 프로그램의 uniform인지 설정해야 한다.
- 사용 예)
 - 변수 선언: 변수 선언 시 타입, 이름과 함께 uniform을 추가한다.
 - 예) `uniform vec4 outColor;`
 - 사용되지 않는 uniform 변수는 오류를 발생시킬 수 있다.
 - 값 가져오기: uniform 변수의 이름을 사용하여 위치(index)를 가져와서 값을 수정 가능
 - `GLuint glGetUniformLocation (GLuint program, const GLchar *name);`
 - uniform 변수의 위치를 가져온다.
 - program: 프로그램 이름
 - name: uniform 변수 이름
 - 리턴값: uniform 변수 위치 (-1: 위치를 찾지 못함)
 - `void glUniform{1|2|3|4}(f|i|ui) (GLuint location, {GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3});`
 - 현재 프로그램에서 uniform 변수의 값을 명시
 - location: 수정할 uniform 변수의 위치
 - v0, v1, v2, v3: 사용될 uniform 변수 값
 - `void glUniformMatrix{2|3|4}(f|i|ui): mat{2|3|4} 전달`

GLSL 기본 문법

- 사용 예) uniform 변수를 사용하여 색상 설정하기
 - 프래그먼트 셰이더에서 uniform 변수 선언

```
#version 330 core
out vec4 FragColor;
```

```
uniform vec4 outColor;
```

//--- 메인 프로그램에서 변수 설정

```
void main()
{
    FragColor = outColor;
}
```

- 메인 프로그램에서 변수에 값 지정

```
float r=0.2, g=0.3, b=0.7;
```

```
glUseProgram (shaderProgram);
```

//--- 사용 셰이더 프로그램 설정

//--- shaderProgram에서 "outColor"라는 이름의 uniform 변수의 위치를 가져온다.

```
int vColorLocation = glGetUniformLocation (shaderProgram, 'outColor');
```

//--- vColorLocation에 (outColor) 이름의 uniform 변수의 값에 r, g, b, 1.0값을 저장

```
glUniform4f (vColorLocation, r, g, b, 1.0);
```

GLSL 기본 문법

- GLSL에 이미 지정되어 있는 built-in 변수들
 - 버텍스 셰이더:
 - `gl_Position`: 버텍스 셰이더가 계산한 정점의 최종 위치, 데이터 타입은 `vec4`
 - 사용 예)

```
void main()
{
    gl_Position = vec4(position, 1.0);
}
```
 - `gl_VertexID`: 현재 처리하고 있는 정점의 ID, 데이터 타입은 `int`
 - 이 인덱스를 사용하여 각 버텍스에 대해 다른 위치를 지정할 수 있다.
 - 예를 들어, `glDrawArrays()`로 3개의 정점을 그리면
 - 첫 번째 정점 → `gl_VertexID = 0`
 - 두 번째 정점 → `gl_VertexID = 1`
 - 세 번째 정점 → `gl_VertexID = 2`
 - 프래그먼트 셰이더
 - `gl_FragCoord`: 현재 프래그먼트의 화면 좌표값, 데이터 타입은 `vec4`
 - `gl_FrontFacing`: 현재 프래그먼트가 앞면에서 생성되었는지 알려주는 `bool` 값

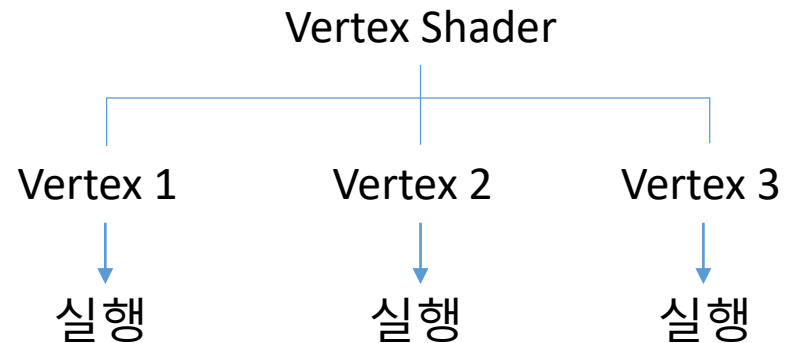
GLSL 기본 문법

- GLSL에 저장되어 있는 built-in 함수들
 - 삼각함수
 - `sin, cos, tan`
 - `asin, acos, atan`
 - 수학함수
 - `pow, log, exp, sqrt, abs, max, min, round, mod, clamp, mix, step, smoothstep`
 - `mix (a, b, t):  $a + t(b-a)$`
 - `step (limit, a): 0 when  $a < \text{limit}$ , 1 when  $a > \text{limit}$`
 - `smoothstep (a0, a1, b): 0 when  $b < a0$ , 1 when  $b > a1$`
 - 기하학 계산 함수
 - `length, distance, dot product, cross product, normalize`
- 함수
 - 함수의 선언 및 정의는 C 문법과 거의 동일하게 지원, 사용자 정의 함수를 만들 수 있다.
 - C 문법과 거의 동일, 함수 overloading이 가능, 재귀 호출 할 수 없다, 포인터 타입이 없어 인자는 모두 call-by-value 형태로 전달
- 그 외
 - 조건문, 반복문, 배열, 구조체 등은 C 문법과 거의 동일하게 지원됨

Shader 코드 구조

- 셰이더 실행 시점은

- C 프로그램의 main ()은 CPU 프로그램의 시작점
- GLSL의 main() 은 각 셰이더 실행 단위에서 실행되는 시작점
 - 예) 버텍스 셰이더라면 각 정점에 대해 실행된다.
 - 버텍스 1 → 버텍스 셰이더 실행 → main ()
 - 버텍스 2 → 버텍스 셰이더 실행 → main ()
 - 버텍스 3 → 버텍스 셰이더 실행 → main ()



- GPU는 각 정점(vertex)에 대해 Vertex Shader를 실행한다.
 - 여러 정점의 Vertex Shader 실행은 GPU의 병렬 처리 구조를 이용하여 동시에 처리될 수 있다.

- Vertex (정점): 3차원 물체를 구성하는 기하학적 점
- Pixel (픽셀): 화면을 구성하는 실제 색상 점

Shader 코드 구조

- 프로그램 구조

```
#version version_number core           //--- 사용 버전 선언

in type in_variable_name;              //--- 변수 선언
out type out_variable_name;
uniform type uniform_variable_name;

void main(void)                         //--- 메인 함수: 인자값 없음
{
    //--- 입력 값을 처리
    //--- 필요한 그래픽 작업 수행
    ...
    //--- 출력 변수 저장
    out_variable_name = output data;
}
```

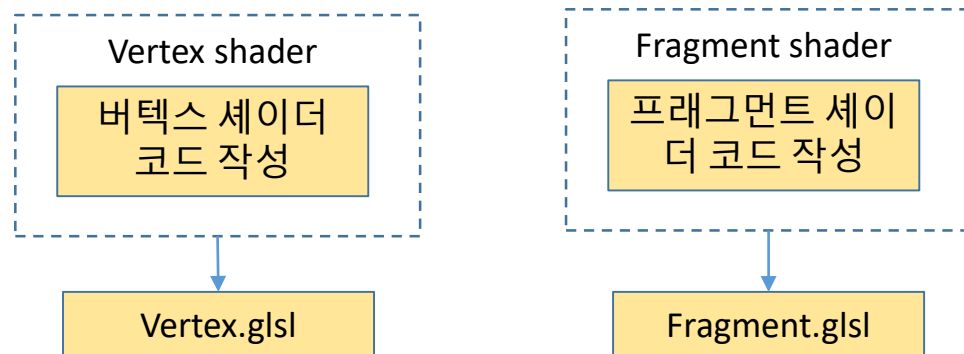
- 오픈지엘의 두 가지 프로파일

- 코어 프로파일 (core): 기존의 삭제된 legacy function 을 사용하지 않음
- 호환성 프로파일 (compatibility): 오픈지엘의 모든 버전과 하위 호환성 유지

셰이더 프로그램 만들기

- 셰이더 코드 만들기

- 셰이더 코드 작성: 필요한 셰이더 코드 작성 (메인 함수와 분리된 텍스트 파일)
 - 버텍스 셰이더와 프래그먼트 셰이더 메인 코드 작성 → 버텍스 셰이더 메인 코드, 프래그먼트 셰이더 메인 코드 작성
 - 각각 vertex.glsl과 fragment.glsl 로 코드 저장



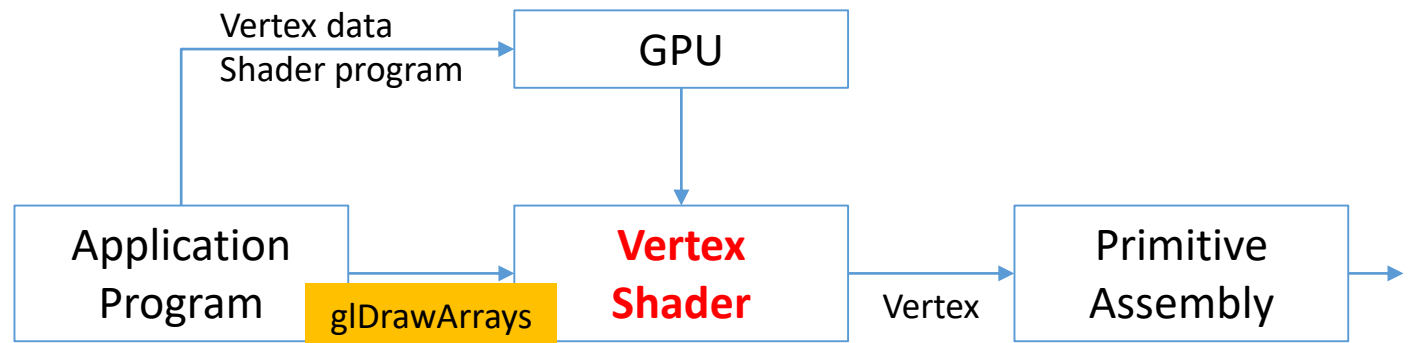
Shader 코드

- **Vertex Shader:** 버텍스를 배치하여 객체의 위치를 정한다.
 - 개별 정점의 처리를 하는 렌더링 파이프라인의 프로그래밍 가능한 첫번째 셰이더 단계 → 각 정점을 최종 화면에 표시될 위치로 변환하는 역할
 - 그리기 명령에 의해 정점 배열 객체에서 지정된 정점 속성 데이터가 제공된다.
 - 정점 위치 변환, 법선 벡터 변환
 - 텍스처 좌표 설정

```
//--- 버텍스 셰이더 예)
#version 330 core

in vec3 vPos;

void main ()
{
    gl_Position = vec4 (vPos.x, vPos.y, vPos.z, 1.0);
}
```



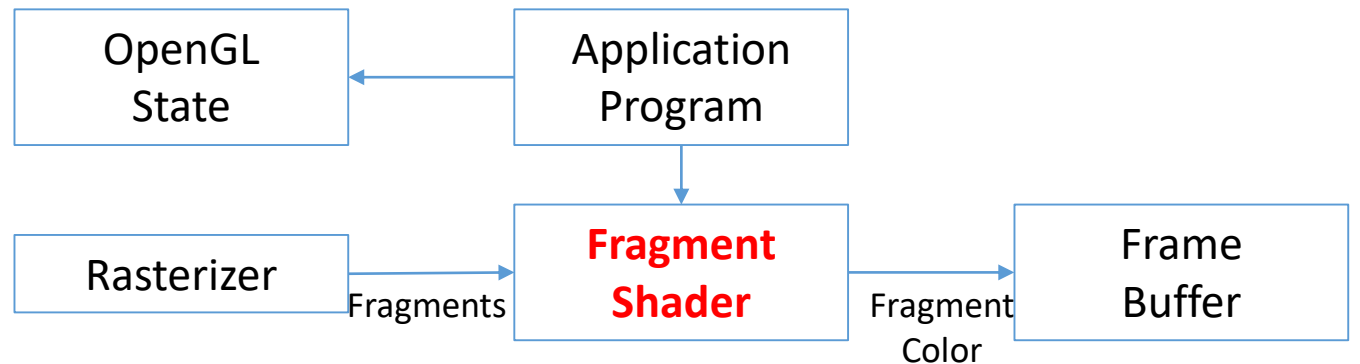
Shader 코드

- **Fragment Shader:** 픽셀의 색상을 정하고 프레임 버퍼로 그 값을 전달한다.
 - Rasterizer에서 생성된 프래그먼트를 처리 → 각 프래그먼트의 최종 색상을 결정
 - 프래그먼트 셰이더는 단일 프래그먼트를 입력으로 사용하고 단일 프래그먼트를 출력으로 생성
 - 최종 색상 설정, 텍스처 색상 적용
 - 조명 설정, 그림자 처리, 투명도 처리

```
//--- 프래그먼트 셰이더 예)
#version 330 core

out vec4 FragColor;

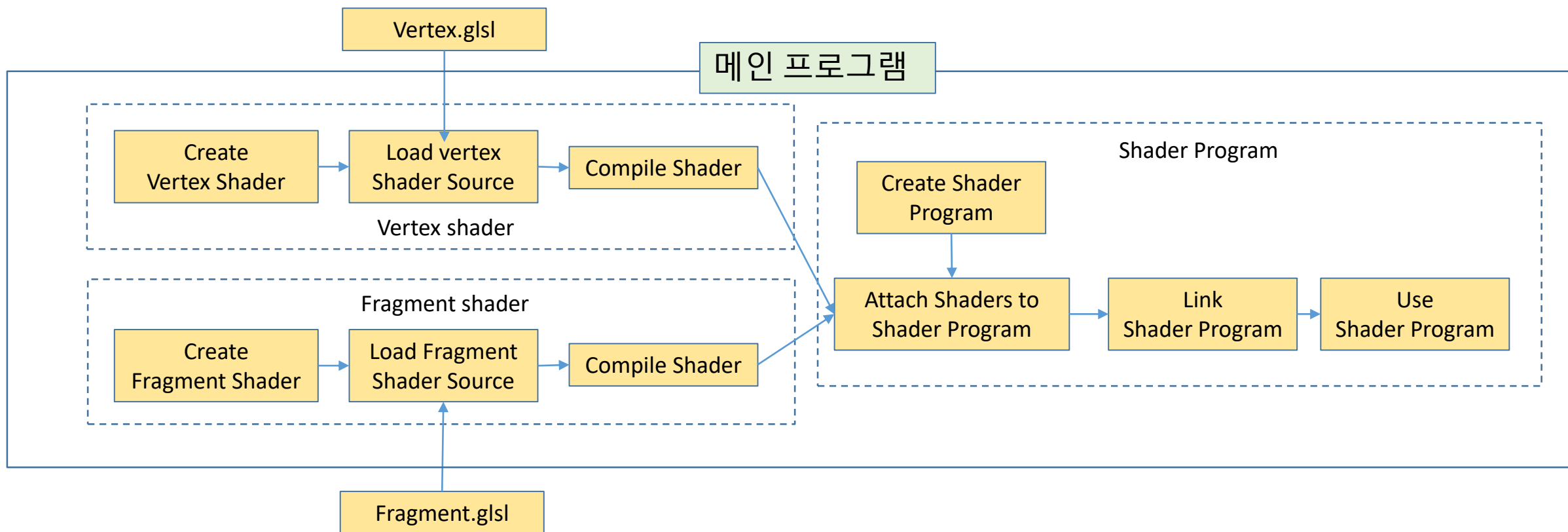
void main ()
{
    FragColor = vec4 (1.0, 0.0, 0.0, 1.0);
}
```



Shader 처리과정

- **메인 프로그램:**

- 기존의 main 함수 및 그 외 함수들
- 분리된 텍스트 파일인 셰이더 코드들을 읽어와 컴파일하고 링크하여 셰이더 프로그램을 생성한다.
- 셰이더 프로그램을 호출하여 그리기, 변환 등을 실행한다.



셰이더 프로그램 만들기

- 셰이더 프로그램 만들기: 메인 프로그램에서 오픈지엘 함수를 호출하여 셰이더 프로그램을 만든다.

1) 셰이더 객체 만들기: 작성한 버텍스 셰이더, 프래그먼트 셰이더 코드로 셰이더 객체 만들기

- Shader 생성 → 버텍스 셰이더 객체와 프래그먼트 셰이더 객체 만들기: **glCreateShader**
- Shader 소스 지정 → 버텍스 셰이더와 프래그먼트 셰이더 코드를 각각 객체에 붙이기: **glShaderSource**
- Shader 컴파일 → 버텍스 셰이더 객체와 프래그먼트 셰이더 객체 컴파일하기: **glCompileShader**

2) 셰이더 프로그램 만들기: 버텍스 셰이더 객체와 프래그먼트 셰이더 객체를 링크하여 셰이더 프로그램 만들기

- Shader Program 생성 → 셰이더 객체를 링크 할 셰이더 프로그램 생성: **glCreateProgram**
- 두 Shader를 Program에 연결 → 버텍스셰이더와 프래그먼트셰이더 객체를 셰이더 프로그램에 붙이기: **glAttachShader**
- Program 링크 → 셰이더 프로그램을 링크: **glLinkProgram**

3) 셰이더 프로그램 활성화: 링크된 셰이더 프로그램 사용하도록 셰이더 프로그램 활성화 하기

- Shader Program 사용 → 셰이더 객체들이 링크된 셰이더 프로그램을 활성화: **glUseProgram**

- Vertex Shader → 하나의 셰이더
- Fragment Shader → 하나의 셰이더
- Shader Program → 여러 셰이더를 연결(link)한 실행 프로그램
- 즉, 버텍스 셰이더 + 프래그먼트 셰이더 → 셰이더 프로그램

셰이더 프로그램 만들기

- 메인 프로그램에서: 오픈지엘 함수를 사용하여 셰이더 프로그램을 만든다.

```
void makeShaderProgram () {  
    glCreateShader (GL_VERTEX_SHADER);           //--- 1. Vertex Shader 생성  
    glShaderSource ();                             //--- 2. Vertex Shader 소스 지정  
    glCompileShader ();                           //--- 3. Vertex Shader 컴파일  
  
    glCreateShader (GL_FRAGMENT_SHADER);          //--- 4. Fragment Shader 생성  
    glShaderSource ();                             //--- 5. Fragment Shader 소스 지정  
    glCompileShader ();                           //--- 6. Fragment Shader 컴파일  
  
    glCreateProgram ();                            //--- 7. Shader Program 생성  
    glAttachShader ();                             //--- 8. 두 Shader를 Program에 연결  
    glLinkProgram ();                             //--- 9. Program 링크  
  
    glUseProgram ();                              //--- 10. Shader Program 사용  
}
```

메인 프로그램에서 따로 함수를 하나 만들어서 셰이더 프로그램 만드는 과정을 넣었음

- Shader는 GLSL로 작성한다.
- 각 Shader를 컴파일한다.
- Vertex Shader와 Fragment Shader를 하나의 Program으로 링크한다.
- glUseProgram()으로 그 셰이더 프로그램을 사용한다.

셰이더 객체 만들기

- 셰이더 객체 만들기: 버텍스 셰이더 객체, 프래그먼트 셰이더 객체를 각각 만든다. → 메인 프로그램에서 오픈지엘 함수를 호출하여 진행
 - 메인 프로그램에서 진행한다

```
GLchar *vertexSource, *fragmentSource;  
GLuint vertexShader, fragmentShader;
```

```
//--- 셰이더 코드 저장할 문자열  
//--- 셰이더 객체
```

```
//--- 버텍스 셰이더 읽어 저장하고 컴파일 하기  
vertexSource = filetobuf ("vertex.glsl " );  
vertexShader = glCreateShader (GL_VERTEX_SHADER);  
glShaderSource (vertexShader, 1, &vertexSource, NULL);  
glCompileShader (vertexShader);
```

```
//--- filetobuf: 사용자 정의 함수로 텍스트를 읽어서 문자열에 저장하는 함수
```

```
//--- 프래그먼트 셰이더 읽어 저장하고 컴파일 하기  
fragmentSource = filetobuf ("fragment.glsl " );  
fragmentShader = glCreateShader (GL_FRAGMENT_SHADER);  
glShaderSource (fragmentShader, 1, &fragmentSource, NULL);  
glCompileShader (fragmentShader);
```

셰이더 프로그램 만들고 활성화 하기

- 만들어진 한 개 이상의 셰이더 객체를 결합하여 한 개의 셰이더 프로그램으로 만든다.
 - 셰이더 객체를 만든 후 셰이더 프로그램을 만든다.

```
GLuint shaderID;  
shaderID = glCreateProgram();
```

```
//--- 셰이더 프로그램 만들기
```

```
glAttachShader (shaderID, vertexShader);  
glAttachShader (shaderID, fragmentShader);
```

```
//--- 셰이더 프로그램에 버텍스 셰이더 붙이기  
//--- 셰이더 프로그램에 프래그먼트 셰이더 붙이기
```

```
glLinkProgram (shaderID);
```

```
//--- 셰이더 프로그램 링크하기
```

- 셰이더 프로그램 활성화

```
glUseProgram (shaderID);
```

함수 프로토타입

- 작성된 셰이더 코드를 응용 프로그램에서 읽어와 런타임시에 셰이더 소스코드를 동적으로 컴파일
 - 셰이더 객체 생성
 - GLuint **glCreateShader** (GLenum shaderType);
 - 빈 셰이더 객체를 생성하여 리턴한다.
 - shaderType: 생성할 셰이더 타입
 - GL_VERTEX_SHADER, GL_FRAGMENT_SHADER, GL_COMPUTE_SHADER, GL_TESS_CONTROL_SHADER, GL_TESS_EVALUATION_SHADER, GL_GEOMETRY_SHADER
 - 리턴 값: 셰이더 오브젝트
 - 셰이더 코드 읽어오기
 - void **glShaderSource** (GLuint shader, GLsizei count, const GLchar **string, const GLint *length);
 - 셰이더 소스코드를 셰이더 객체로 전달해서 복사본을 유지한다.
 - shader: 셰이더 오브젝트
 - count: string 배열의 구성요소의 개수
 - string: 소스코드가 저장되어 있는 배열 이름
 - length: 소스코드 크기
 - 셰이더 컴파일
 - void **glCompileShader** (GLuint shader);
 - 셰이더 객체에 포함된 소스코드를 컴파일한다.
 - shader: 컴파일 할 셰이더 객체

함수 프로토타입

- 여러 셰이더를 결합하여 한 개의 셰이더 프로그램으로 링크
 - 셰이더 프로그램 만들기
 - GLuint **glCreateProgram** ();
 - 셰이더 객체에 붙일 프로그램 객체를 생성한다.
 - 셰이더 객체들을 프로그램에 첨부
 - void **glAttachShader** (GLuint program, GLuint shader);
 - 셰이더 객체를 프로그램 객체에 붙인다.
 - program: 셰이더를 붙일 프로그램 객체
 - shader: 셰이더 객체
 - 셰이더 프로그램 링크
 - void **glLinkProgram** (GLuint program);
 - 프로그램 객체에 붙인 모든 셰이더 객체를 링크한다.
 - program: 링크할 프로그램 객체
 - 셰이더 객체 삭제하기
 - void **glDeleteShader** (GLuint shader);
 - 셰이더 객체를 삭제한다. 셰이더가 프로그램 객체에 링크되면, 프로그램이 바이너리 코드를 보관하며 셰이더는 더 이상 필요없게 된다.
 - shader: 삭제 할 셰이더

함수 프로토타입

- 프로그램 활성화
 - 셰이더 프로그램 사용
 - `void glUseProgram (GLuint program);`
 - 현재 렌더링 상태에 프로그램 객체를 활성화한다.
 - program: 실행 할 프로그램

함수 프로토타입

- 셰이더 상태 가져오기

- void **glGetShaderiv** (GLuint shader, GLenum pname, GLint *params);

- 셰이더 정보 가져오기

- shader: 셰이더 객체

- pname: 객체 파라미터

- GL_SHADER_TYPE, GL_DELETE_STATUS, GL_COMPILE_STATUS, GL_INFO_LOG_LENGTH, GL_SHADER_SOURCE_LENGTH)

- params: 리턴 값

- GL_SHADER_TYPE: 셰이더 타입 리턴 (GL_VERTEX_SHADER / GL_FRAGMENT_SHADER)

- GL_DELETE_STATUS: 셰이더가 삭제됐으면 GL_TRUE, 아니면 GL_FALSE

- GL_COMPILE_STATUS: 컴파일이 성공했으면 GL_TRUE, 아니면 GL_FALSE

- GL_INFO_LOG_LENGTH: 셰이더의 INFORMATION LOG 크기

- GL_SHADER_SOURCE_LENGTH: 셰이더 소스 크기

- 에러 발생 시, GL_INVALID_VALUE, GL_INVALID_OPERATION, GL_INVALID_ENUM

- void **glGetShaderInfoLog** (GLuint shader, GLsizei maxLength, GLsizei *length, GLchar *infoLog);

- 셰이더 객체의 information log 가져오기

- shader: 셰이더 객체

- maxLength: information log 크기

- length: infoLog 길이

- infoLog: information log

함수 프로토타입

- 프로그램 상태 가져오기
 - void **glGetProgramiv** (GLuint program, GLenum pname, GLint *params);
 - 프로그램 객체 정보 가져오기
 - program: 프로그램
 - pname: 객체 파라미터
 - GL_DELETE_STATUS, GL_LINK_STATUS, GL_VALIDATE_STATUS, GL_INFO_LOG_LENGTH, GL_ATTACHED_SHADERS, GL_ACTIVE_ATOMIC_COUNTER_BUFFERS, GL_ACTIVE_ATTRIBUTES, GL_ACTIVE_ATTRIBUTE_MAX_LENGTH, GL_ACTIVE_UNIFORMS, GL_ACTIVE_UNIFORM_BLOCKS, GL_ACTIVE_UNIFORM_BLOCK_MAX_NAME_LENGTH, GL_ACTIVE_UNIFORM_MAX_LENGTH, GL_COMPUTE_WORK_GROUP_SIZE, GL_PROGRAM_BINARY_LENGTH, GL_TRANSFORM_FEEDBACK_BUFFER_MODE, GL_TRANSFORM_FEEDBACK_VARYINGS, GL_TRANSFORM_FEEDBACK_VARYING_MAX_LENGTH, GL_GEOMETRY_VERTICES_OUT, GL_GEOMETRY_INPUT_TYPE, and GL_GEOMETRY_OUTPUT_TYPE.
 - params: 리턴 값
 - GL_DELETE_STATUS, GL_LINK_STATUS, GL_VALIDATE_STATUS, GL_ATTACHED_SHADERS...
 - void **glGetProgramInfoLog** (GLuint program, GLsizei maxLength, GLsizei *length, GLchar infoLog);
 - 프로그램 객체 information log 가져오기
 - program: 프로그램
 - maxLength: information log 크기
 - length: infoLog 길이
 - infoLog: information log
- 이 함수들은 GL 2.0 이상에서 지원됨

셰이더 사용하여 도형 그리기

- 셰이더 프로그램을 만든 후, 셰이더 프로그램을 사용하여 도형을 그린다.

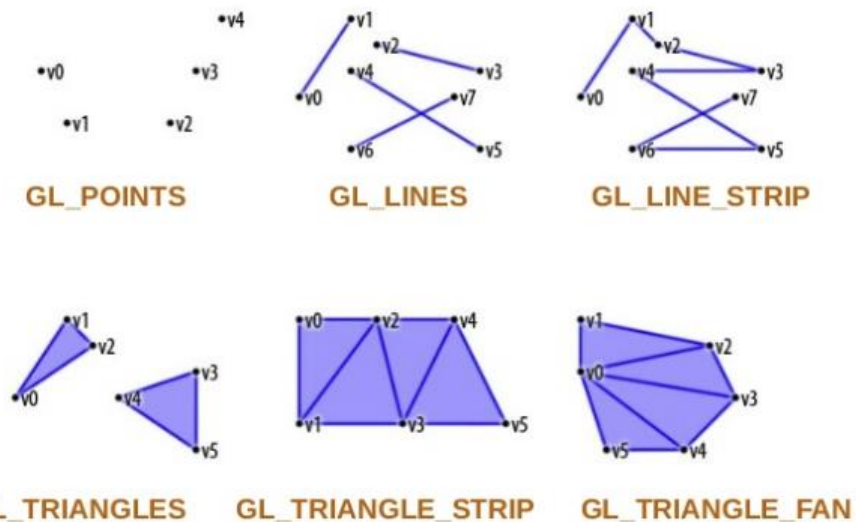
```
void DrawScene ()  
{  
    //--- 필요한 작업 진행  
  
    glUseProgram (shaderProgramID);  
  
    glDrawArrays (GL_TRIANGLES, 0, 3);      //--- 도형 그리기  
  
    glutSwapBuffers ();  
}
```

함수 프로토타입

- 배열 데이터로부터 프리미티브 렌더링
 - `void glDrawArrays (GLenum mode, GLint first, GLsizei count);`
 - 배열 데이터로부터 프리미티브 렌더링 하기
 - mode: 렌더링 할 프리미티브의 종류
 - GL_POINTS, GL_LINE_STRIP, GL_LINE_LOOP, GL_LINES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN, GL_TRIANGLES 등
 - first: 배열에서 도형의 시작 인덱스
 - count: 렌더링 할 인덱스 개수
 - 프리미티브 (primitive): 오픈지엘 렌더링의 기본 단위로 이용 가능한 가장 단순한 요소 → 점, 선, 삼각형

- Primitive 타입

- Points: [GL_POINTS](#)
- Line
 - Lines: [GL_LINES](#)
 - Line strips: [GL_LINE_STRIP](#)
 - Line loops: [GL_LINE_LOOP](#)
- Triangle
 - Triangles: [GL_TRIANGLES](#)
 - Triangle strips: [GL_TRIANGLE_STRIP](#)
 - Triangle fans: [GL_TRIANGLE_FAN](#)



함수 프로토타입

- GL에서 그리기
 - void **glDrawElements** (GLenum mode, GLsizei count, GLenum type, const GLvoid *indices);
 - 배열 데이터로부터 프리미티브 렌더링 하기, 배열 데이터의 인덱스를 사용
 - mode: 렌더링 할 프리미티브의 종류
 - GL_POINTS, GL_LINE_STRIP, GL_LINE_LOOP, GL_LINES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN, GL_TRIANGLES 등
 - count: 렌더링할 요소의 개수
 - type: indices 값의 타입
 - GL_UNSIGNED_BYTE, GL_UNSIGNED_SHORT, 또는 GL_UNSIGNED_INT
 - indices: 바인딩 되는 버퍼의 데이터 저장소에 있는 배열의 첫 번째 인덱스 오프셋

함수 프로토타입

- 기본 속성 바꾸기
 - void **glPointSize** (GLfloat size);
 - 점 크기 설정
 - size: 점의 크기 (초기값: 1)
 - void **glLineWidth** (GLfloat width);
 - 선의 굵기 조정
 - width: 선의 굵기 (초기값: 1)
 - void **glPolygonMode** (GLenum face, GLenum mode);
 - 폴리곤 모드 설정
 - face: 모드를 설정할 면 (GL_FRONT_AND_BACK)
 - mode: 모드
 - GL_POINT, GL_LINE, GL_FILL